

How the pipeline works

How a study conversation becomes a tracked mastery record — four layers, zero daemons.

The four layers at a glance

Everything the system does happens in one of four places. They communicate only through plain files on disk.

1 · LAYER

INSTRUCTIONS

Markdown files that tell the LLM who it is and what rules to follow. Read at session start — never executed, never modified.

CLAUDE.md AGENTS.md SKILL.md

2 · LAYER

HOOKS

Shell scripts Claude Code fires automatically at lifecycle moments. They trigger work — they never do it themselves.

session-start.py session-end.py
pre-compact.py

3 · LAYER

SCRIPTS

Python processes that do the real work: extract, validate, compile, regenerate.

flush.py ingest.py compile.py
validate.py

4 · LAYER

DATA

The only persistent state. Source material, session evidence, compiled knowledge.

weeks-lessons/ sessions/ inbox/
knowledge/

How they connect

HOOK fires → .md loaded → SCRIPT runs → DATA updated

Each layer has one responsibility. Instructions never execute. Hooks never process data. Scripts never teach. Data never decides. When something breaks, you know exactly which layer to open.

The core principle

Every session eventually becomes a structured evidence file. Mastery ratings only change when there is evidence from an actual conversation to justify the change. Nothing is assumed. Nothing is guessed.

The hook layer

Hooks are Claude Code's lifecycle triggers. They are small Python scripts — not workers. They fire, do minimal setup, hand off to a script, and return immediately.

STEP 1

SessionStart

Fires: when you open Claude Code in this project.

Job: Inject a compact status header into the session context.

Reads: `knowledge/progress.md` `sessions/` `inbox/`

Writes: Nothing written to disk — echoes JSON that Claude Code splices into context.

Why: Without this, every session starts blind. With it, Claude knows the current week, days to exams, stale concepts, and pending inbox files.

STEP 2

SessionEnd

Fires: when you close Claude Code or the session times out.

Job: Trigger the flush pipeline. Spawns `flush.py` as a detached background process.

Reads: `~/ .claude/projects/<slug>/*.jsonl` (raw conversation log)

Writes: A temp context file on disk. `flush.py` runs independently after the hook returns.

Why: `flush.py` runs an LLM call that takes 20–40 seconds. The hook returns immediately so Claude Code doesn't block waiting.

STEP 3

PreCompact

Fires: before Claude Code compresses the context window.

Job: Same as `SessionEnd`, but higher threshold (≥ 5 turns vs ≥ 2).

Reads: Same as `SessionEnd`.

Writes: Same as `SessionEnd`.

Why: Compaction discards raw turns. Without this, a long mid-session conversation would evaporate without ever becoming evidence.

The recursion guard

Every spawned script sets `CLAUDE_INVOKED_BY=memory_flush` before importing anything that might re-invoke Claude. Without this, `flush.py`'s own LLM call could re-trigger the `SessionEnd` hook — infinite recursion. One environment variable prevents a catastrophic bug.

Hooks are triggers. Scripts are workers.

You can run `./flush-now.sh` manually without Claude Code being open at all. The hook and the script are decoupled — that's the point.

The script layer

Four Python scripts do all the real work. Each has a single input, a single output, and hands off to the next via files on disk.

SCRIPT

flush.py · Extract structured evidence from a raw Claude transcript.

INPUT Raw JSONL conversation turns (last 120). Week context and existing concept slugs passed alongside.

OUTPUT One session-log markdown file under `sessions/week-NN/` with strict YAML frontmatter + narrative body.

SCRIPT

ingest.py · Normalize freeform inbox entries into canonical session logs.

INPUT Markdown files in `inbox/` — Codex notes, voice memos, ChatGPT exports. Minimal or messy frontmatter.

OUTPUT Canonical session logs in `sessions/week-NN/`. Originals move to `inbox/processed/` or `inbox/failed/`.

SCRIPT

compile.py · Merge session evidence into per-concept mastery files; regenerate aggregates.

INPUT All session logs under `sessions/` that haven't been compiled yet (tracked by content hash in `state.json`).

OUTPUT Per-concept files in `knowledge/concepts/`, week aggregates, `index.md`, `progress.md`, `log.md`.

SCRIPT

validate.py · Pydantic schema enforcement for every YAML frontmatter the pipeline writes.

INPUT Any markdown file with frontmatter: inbox headers, session logs, concept files.

OUTPUT Hard-fail with precise *path* → *field* → *error* on canonical files. Forgiving normalization on inbox (aliases like `understanding` → `understand`).

Strictness is asymmetric by design

Inbox normalization is forgiving — external LLMs produce messy output and it is easier to clean up than retrain. Canonical session logs and concept files are strict — bad data here corrupts the mastery record permanently. Validation hard-fails at every boundary where data enters the authoritative zone.

The knowledge directory

knowledge/ is the only place that tracks mastery state. Everything inside is derived — rebuilt by `compile.py` from session evidence. Never edited by hand.

FILE

knowledge/concepts/<slug>.md

One file per concept. Contains a YAML frontmatter snapshot: Bloom-level ratings, last studied/revised dates, weak spots, solid connections, and a probe history cache.

FILE

knowledge/weeks/week-NN.md

Rolled-up summary of every concept from that week. Instead of opening 6 concept files, open one and see the week's status at a glance. Regenerated every compile run.

FILE

knowledge/index.md

Master table of every concept: slug, title, week, last revised, ratings at a glance. This is what SessionStart reads to find stale concepts to surface.

FILE

knowledge/progress.md

Tiered progress dashboard. Coverage per week at each Bloom level. Weak-spot rollup across all concepts. Injected into every session by SessionStart.

The evidence rule — how ratings change

For every concept, for every Bloom level, on every compile run:

Prior rating	Session showed	New rating
good / solid	weak / mid	drops — evidence of stumble
weak / mid	good / solid	rises — evidence of depth
anything	untested	stays — level not probed
anything	same as prior	stays — no new information

Probe history — the deterministic cache

Concept files carry a `probe_history` block. After every compile run, a Python function (not the LLM) rebuilds it from scratch by scanning all session logs:

1. Scan all sessions for probes referencing this concept.
2. Keep unresolved *weak/mid* probes — unless a later unprompted *good/solid* at the same Bloom level supersedes them.
3. Cap at 5 entries total (`PROBE_HISTORY_LIMIT`).

Why deterministic, not LLM-driven

The LLM decides whether a rating changes — that requires reading comprehension. The list of probes to keep is pure set-logic — no judgment needed. Using an LLM here would introduce cache drift every rewrite. Python rebuilds it identically from the same input, every time. Right tool, right job.

Sessions, source material & the full loop

Source material — weeks-lessons/

Read-only. Never touched by the pipeline. Raw lectures, readings, and transcripts for each week live here. At the start of a learn session, Claude reads this directory to build the overview map. Immutable source of truth.

The inbox — inbox/

Staging area for external sessions — Codex notes, voice memos, ChatGPT exports. Anything that didn't come from a live Claude Code session lands here. `ingest.py` normalizes them and moves them to `sessions/`.

DIR inbox/ Incoming — messy, forgiving schema.	DIR inbox/processed/ Successfully ingested.	DIR inbox/failed/ Failed — <code>.error.txt</code> sibling explains why.
---	--	---

Session logs — sessions/week-NN/

The evidence trail. Each file is the output of `flush.py` or `ingest.py` — a compressed, structured record of one session. Not the raw transcript. Just: what was demonstrated, what stuck, what to revisit.

End-to-end timeline — one complete session

T+0	HOOK	Student opens Claude Code. <code>session-start.py</code> fires — injects week, progress, stale concepts. Claude reads <code>CLAUDE.md</code> and <code>SKILL.md</code> .
T+1m	LLM	Student: “let's learn X.” Claude reads <code>weeks-lessons/week-NN/</code> , builds overview map.
T+20m	LLM	Session runs — overview, student-chosen path, relational probes at every Bloom level. Phase 5 summary.
T+35m	HOOK	Student closes Claude Code. <code>session-end.py</code> fires. Extracts last 120 turns to temp file.
T+35m	SCRIPT	<code>flush.py</code> spawned detached. Runs an LLM call against the transcript context.
T+36m	DATA	<code>sessions/week-01/2026-04-23-learn-X.md</code> written with structured YAML frontmatter.
T+37m	SCRIPT	If past 6 pm, <code>flush.py</code> auto-spawns <code>compile.py</code> . <code>validate.py</code> hard-fails or passes every session log first.
T+38m	SCRIPT	<code>compile.py</code> merges evidence into <code>knowledge/concepts/X.md</code> via the evidence rule.
T+38m	DATA	<code>rebuild_probe_history()</code> scans all sessions. <code>regenerate_progress()</code> rebuilds dashboard.
T+39m	HOOK	Next <code>session-start.py</code> shows the updated progress header. Cycle closes.

One session, four layers, zero coordination overhead

No queue, no database, no daemon. Each layer reads what's on disk, writes what's on disk, and returns. State lives in files. Everything is crash-safe by default — if any step fails, rerunning it picks up exactly where it left off because the previous step's output is already persisted.